

Exercício (Novo): API de Finanças — "Meu Controle de Gastos"

Turma: LionsDev

Tópicos: Boilerplate, rotas protegidas com JWT, dono do recurso via token, validação de valor no service, **cálculo de resumo em JavaScript** (entradas, saídas e saldo), Query Params e status codes.

Nível: intermediário. A novidade aqui é uma rota de **resumo** que calcula totais a partir das suas transações.

1. Contexto

Você vai criar uma API de **controle financeiro pessoal**. Cada usuário registra suas **entradas** (salário, vendas) e **saídas** (contas, compras), e pode pedir um **resumo** com total de entradas, total de saídas e o **saldo**. Cada pessoa só enxerga o próprio dinheiro.

Para este exercício, trabalhe com valores em **reais** usando **Number** (ex.: **150.5**). Não precisa usar centavos.

2. Ponto de Partida: o Boilerplate

Partimos do **boilerplate LionsDev**: <https://github.com/nicolassmotta/lionsdev-boilerplate>

1. Clone, `npm install`, crie o `.env` (`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`) e suba o servidor.
2. Já estão prontos: camada de `Usuario`, cadastro/login com `bcrypt/JWT`, middleware `autenticar` (preenche `req.usuario = { id, email }`), `criarErro` e o middleware de erro.

Antes de começar, pegue seu token e envie `Authorization: Bearer SEU_TOKEN` em todas as rotas novas.

3. Regras de Ouro

1. **Toda rota é protegida.** `router.use(authenticar)` no topo do arquivo de rotas.
2. **O dono vem do token** (`req.usuario.id`), **nunca do body**.
3. **Toda consulta filtra pelo dono** (`{ _id: idTransacao, usuario: idDoUsuario }`).
Se não achar → **404** .
4. **Listagem nunca dá 404:** sem transações, devolva array vazio (e o resumo vem zerado).

4. Arquivos que você vai criar

Camada	Arquivo
Model	<code>src/models/transacao.model.js</code>
Repository	<code>src/repositories/transacao.repository.js</code>
Service	<code>src/services/transacao.service.js</code>
Controller	<code>src/controllers/transacao.controller.js</code>
Routes	<code>src/routes/transacao.routes.js</code>

5. Etapa 1 — Model **Transacao**

- `descricao`: `String`, obrigatório, `trim` .
- `tipo`: `String`, obrigatório, `enum`: `["entrada", "saida"]` .
- `categoria`: `String`, opcional, `trim` (ex.: `"salário"`, `"mercado"`, `"lazer"`).
- `valor`: `Number`, obrigatório, `min`: `[0.01, "0 valor deve ser maior que zero."]` .
- `data`: `String`, opcional (ex.: `"2026-06-17"`).
- `usuario`: `ObjectId`, `ref`: `"Usuario"`, obrigatório.
- `timestamps`: `true` .

Critérios de aceite: `descricao`, `tipo`, `valor` e `usuario` são obrigatórios; `tipo` só aceita `entrada` ou `saida`; `valor` não aceita zero nem negativo.

Conceito novo — `enum`, `min` numérico e o campo de dono (`ObjectId` + `ref`)

- `enum` trava `tipo` a uma lista fechada (`"entrada"` / `"saida"`); qualquer outro valor vira **400** .
- `min` valida um número mínimo já no Schema (sem `if`): `valor`: `{ type: Number, required: true, min: [0.01, "0 valor deve ser maior que zero."] }` .
- `usuario` guarda o **dono** da transação — uma referência ao `_id` de um `Usuario` :

```
import mongoose from "mongoose";
// ...dentro do Schema
tipo: { type: String, required: true, enum: ["entrada", "saida"] },
valor: { type: Number, required: true, min: [0.01, "O valor deve ser maior que zero."] },
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario", required: true },
```

6. Etapa 2 — Repository

O repository é a **única** camada que fala com o Mongoose. Antes de escrever, pense em quais operações cada rota vai exigir e projete as funções para:

- Criar uma transação.
- Listar as transações **do dono**, deixando espaço para um filtro extra opcional (usaremos no bônus) e ordenando da mais recente para a mais antiga.
- Buscar, atualizar e remover **uma** transação — sempre **filtrando pelo dono junto do id** (Regra de Ouro nº 3), nunca só pelo `_id`.
- Exporte tudo num objeto `TransacaoRepository`.

Critérios de aceite: buscar/atualizar/remover com o id de uma transação de outra pessoa **não encontra nada** (vira `404` no service); a listagem aceita um filtro extra sem furar o filtro por dono.

Pense antes de codar: por que filtrar pelo dono **já na query** ({ `_id`, `usuario` }) é mais seguro do que buscar só por `_id` e depois conferir o dono num `if`? E o que a API deve responder quando a transação **não existe** e quando ela **é de outra pessoa** — por que as duas viram a mesma resposta?

7. Etapa 3 — Service

Aqui mora a **regra de negócio**. O service precisa cobrir:

- **Registrar** uma transação para o usuário logado. Mesmo com o Schema validando, **reforce na mão** que `valor` é um número maior que zero antes de salvar (defesa em profundidade) e responda `400` se não for. O dono vem do token, **nunca** do body.

- **Listar** as transações do usuário.
- **Buscar, atualizar e remover** uma transação do usuário — quando o repository devolver `null` (não existe **ou** não é sua), lance `404` com `criarErro`. Na remoção bem-sucedida, devolva uma mensagem.
- **Resumo** (`resumoDoUsuario`) — a parte nova: a partir de **todas** as suas transações, calcule, em JavaScript, `totalEntradas`, `totalSaidas`, `saldo` (entradas – saídas) e `quantidade`. Sem transações, tudo zero.

Critérios de aceite: registrar com `valor` 0 ou negativo → `400`; buscar/editar/remover transação que não é sua → `404`; o resumo de quem nunca lançou nada vem com tudo `0` (nunca quebra); `saldo` sempre igual a `totalEntradas - totalSaidas`.

Conceito novo — somar uma lista em JavaScript com `filter` + `reduce`

O banco te devolve um **array**. Para fechar os totais do resumo, você vai filtrar e somar esse array na mão. As duas ferramentas:

- `.filter(fn)` devolve um novo array só com os itens que passam no teste (ex.: só os de um certo **tipo**).
- `.reduce((acumulador, item) => ..., inicial)` "espreme" o array num único valor. O **inicial** é o ponto de partida — começando em `0`, uma lista vazia soma `0` e nunca dá erro.

Exemplo **genérico** (somar os preços de um carrinho) — a ideia, não a resposta pronta:

```
const total = carrinho
  .filter((item) => item.emEstoque)
  .reduce((soma, item) => soma + item.preco, 0);
```

Agora é com você: aplique essa ideia para somar os `valor` por `tipo` e montar o resumo. (Prefere um `for...of`? Também resolve.)

8. Etapa 4 — Controller e Etapa 5 — Routes

Crie o controller (`try/catch` + `next(error)`) e as rotas. No

`src/routes/transacao.routes.js`, com `router.use(authenticar)` no topo:

- `POST /` → registrar (`201`)

- `GET /` → `listarMinhas ( 200 com { transacoes } )`
- `GET /resumo` → `resumo ( 200 )` — **declare antes de** `GET /:id`
- `GET /:id` → `buscarMinha ( 200 )`
- `PATCH /:id` → `atualizarMinha ( 200 )`
- `DELETE /:id` → `removerMinha ( 200 )`

No `src/app.js`, **antes** do middleware 404:

```
import transacaoRoutes from "../routes/transacao.routes.js";
app.use("/api/transacoes", transacaoRoutes);
```

Atenção à ordem das rotas: se `GET /:id` vier antes de `GET /resumo`, o Express vai entender `"resumo"` como um `:id` e quebrar a rota de resumo.

9. Rotas finais

Método	Rota	Proteção	Descrição
POST	<code>/api/transacoes</code>	JWT	Registrar entrada ou saída
GET	<code>/api/transacoes</code>	JWT	Listar minhas transações
GET	<code>/api/transacoes/resumo</code>	JWT	Total de entradas, saídas e saldo
GET	<code>/api/transacoes/:id</code>	JWT	Ver uma transação minha
PATCH	<code>/api/transacoes/:id</code>	JWT	Editar uma transação minha
DELETE	<code>/api/transacoes/:id</code>	JWT	Remover uma transação minha

10. Fluxo de Testes (use o `requests.http`)

1. Cadastro/login → copie o `token`.
2. `POST /api/transacoes` com `{ "descricao": "Salário", "tipo": "entrada", "valor": 3000 }` → `201`.
3. `POST` com `{ "descricao": "Mercado", "tipo": "saida", "valor": 450.5 }` → `201`.
4. `POST` com `{ "descricao": "Erro", "tipo": "saida", "valor": 0 }` → `400`.

5. `GET /api/transacoes/resumo` → **preveja no papel** `totalEntradas`, `totalSaidas` e `saldo` a partir dos lançamentos acima e confira se a API bate.
6. `GET /api/transacoes` → só as suas.
7. `DELETE /api/transacoes/:id` → `200`; repita → `404`.
8. Com um **segundo usuário**, peça `/api/transacoes/resumo` → vem zerado (não enxerga o dinheiro do primeiro).
9. Qualquer rota **sem token** → `401`.

11. Desafios Bônus

1. `GET /api/transacoes?tipo=saida` — filtra **entre as suas** por tipo (use `req.query.tipo`).
2. No resumo, adicione `totalPorCategoria` (um objeto com a soma das saídas por `categoria`).
3. `GET /api/transacoes?de=2026-06-01&ate=2026-06-30` — filtra suas transações por intervalo de `data`.

Como ler um Query Param (`?tipo=saida`): no controller, leia de `req.query` e repasse ao service, **sempre** somando ao filtro do dono para não vaziar dados de outras pessoas:

```
// controller
const filtro = {};
if (req.query.tipo) filtro.tipo = req.query.tipo;
const transacoes = await TransacaoService.listarMinhas(req.usuario.id,
filtro);
// no repository: Transacao.find({ usuario: idUsuario, ...filtro })
```